
Capturing non-Markovian effects in weather prediction models

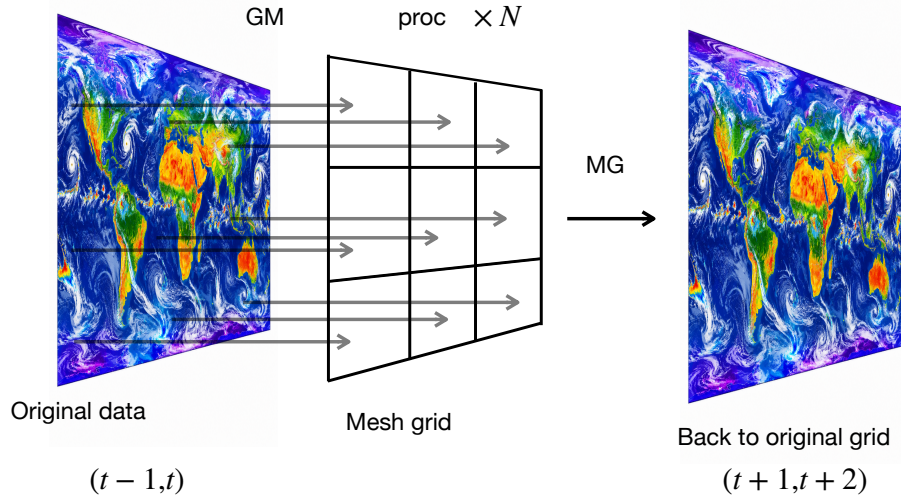
Abstract

Many existing weather forecasting models are Markovian in their chosen state representation, yet unresolved atmospheric processes can make coarse-grained dynamics history dependent. We introduce GraphCast–Mamba, which augments a frozen, pretrained 1° GraphCast model with a correction branch combining graph message passing and persistent temporal memory. On a development set of 32 matched forecast starts, selected configurations reduce the original GraphCast loss by approximately 16 over a ten-day rollout and 23 at day ten. These results highlight the potential of learned non-Markovian corrections for improving long-range weather forecasts.

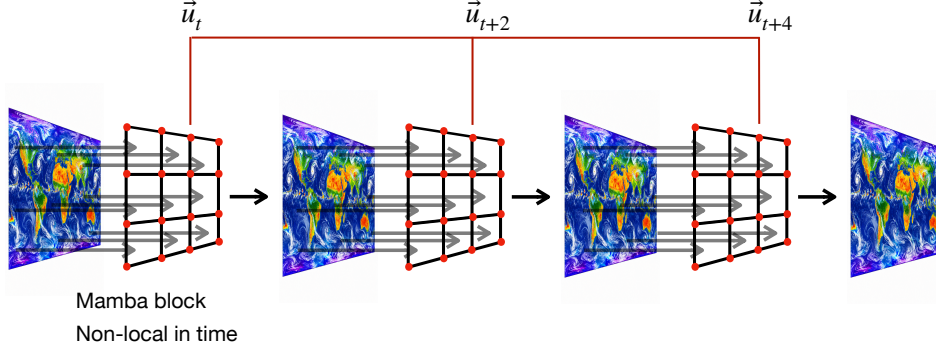
1 Introduction to the architecture

Turbulent flows are known as a notoriously difficult framework for numerical simulations. One of the common method in such simulations is the *direct numerical simulations* (DNS) that discretize the Navier-Stokes equation to a space and time grid and then simulate directly. Such discretizations, however require a very fine grid resolution scaling as $N_{\text{grid}} \sim (Re)^\alpha$ with $\alpha \sim 2.05$ Yang and Griffin [2021]. Insufficient small-scale resolution may leave mean dissipation and enstrophy approximately converged while substantially underestimating high-order moments of velocity gradients, dissipation, and enstrophy, as well as the tails of velocity-gradient distributions and statistics involving second velocity derivatives Donzis et al. [2008]. While this becomes expensive, a variety of models that regularize the equations on a larger scale, introducing proper dissipative terms at the cost of short time accuracy Wilson and Sawford [1996]. Another approach consists in keeping coarse-grained scales, without non-controlled dissipative approximation, but at a cost of keeping the non-Markovian memory terms Lin et al. [2021], De Wit et al. [2026].

In this article, we explore the potential of such methods for a weather prediction models.



(a) GraphCast maps the input grid to a mesh, performs message passing, and decodes back to the grid.



(b) Mamba augments mesh processing with persistent temporal states.

Figure 1: Spatial and temporal processing components. In the experiments, a frozen GraphCast baseline is combined at the output with a separate trainable correction branch containing graph message passing and Mamba.

1.1 A hybrid GraphCast–Mamba architecture

We study whether a persistent latent memory improves forecasts beyond the finite input history supplied to a pretrained GraphCast model. GraphCast uses two successive atmospheric states as input; its forecast is Markovian with respect to this augmented state. Our correction branch retains additional temporal information through Mamba states carried across autoregressive steps. This introduces history dependence relative to the baseline input representation. Figure 1 illustrates the graph and temporal processing components.

GraphCast The baseline is a pretrained GraphCast checkpoint. This graph neural network operates on a latitude–longitude grid and an auxiliary mesh, with graph $\mathcal{G}(\mathcal{V}^G, \mathcal{V}^M, \mathcal{E}^{G2M}, \mathcal{E}^M, \mathcal{E}^{M2G})$. The vertices $\mathcal{V}^G$ represent the meteorological grid and $\mathcal{V}^M$ the mesh. Encoding uses grid-to-mesh edges $\mathcal{E}^{G2M}$, followed by $N = 16$ mesh message-passing steps over $\mathcal{E}^M$ and decoding over mesh-to-grid edges $\mathcal{E}^{M2G}$. Latent node features have width $W_{GC} = 512$. The mesh provides a more compact representation for spatial processing than the meteorological grid.

Mamba Mamba is an attention-free sequence model based on selective state-space models (SSMs), designed to model long temporal dependencies with computation that scales linearly with sequence length Gu and Dao [2023]. A conventional continuous-time SSM maps an input sequence $x(t)$ to a

latent memory state $h(t)$ according to

$$\frac{dh(t)}{dt} = Ah(t) + Bx(t), \quad y(t) = Ch(t) + Dx(t). \quad (1)$$

Here, A is a diagonal matrix that determines how quickly memory decays or evolves, B and C determines how new inputs are written into the state and readout from the state respectively, and D provides a direct input-to-output skip connection. For a single SSM channel, each of the matrices have fixed dimension $d_{\text{state}} = 16$ and Δ - is a scalar. If A , B , and C are fixed, the same linear dynamical system is applied at every time step. For a discrete step size Δ , its exact discretization is

$$h_t = \bar{A}h_{t-1} + \bar{B}x_t, \quad \bar{A} = \exp(\Delta A), \quad \bar{B} = A^{-1}[\exp(\Delta A) - I]B. \quad (2)$$

Mamba makes this SSM *selective*: rather than using fixed input and readout matrices and a fixed step size, it computes B_t , C_t , and Δ_t from the current input. Thus, the model can decide which information to retain, overwrite, or expose at each time step.

In our model, Mamba operates independently on the temporal sequence of each GraphCast mesh node. Let $x_t^{\text{in}} \in \mathbb{R}^{W_{\text{gc}}}$ denote the mesh latent feature at a given node and forecast step. A layer-normalized input is projected into two d_{inner} -dimensional branches,

$$[\tilde{x}_t, z_t] = W_{\text{in}} \text{LN}(x_t^{\text{in}}), \quad \tilde{x}_t, z_t \in \mathbb{R}^{d_{\text{inner}}}. \quad (3)$$

The first branch, $\tilde{x}_t$, supplies the SSM input. The second branch, z_t , is a learned output gate: it does not update the memory state directly, but controls how much of the SSM output is passed onward.

Before entering the SSM, $\tilde{x}_t$ is processed by a depthwise causal one-dimensional convolution with kernel width four:

$$u_{t,d} = \text{SiLU} \left(b_d^{\text{conv}} + \sum_{k=0}^{d_{\text{conv}}-1} w_{d,k}^{\text{conv}} \tilde{x}_{t-k,d} \right), \quad d = 1, \dots, d_{\text{inner}}, \quad (4)$$

where $\text{SiLU}(a) = a/(1 + e^{-a})$. ‘‘Depthwise’’ means that each of the d_{inner} channels has its own d_{conv} learned convolution weights $w_{d,k}^{\text{conv}}$ and does not mix with other channels at this stage. ‘‘Causal’’ means that u_t depends only on the current input and the three previous inputs, never on future inputs. During autoregressive inference, the preceding three projected inputs are retained in a convolution cache, so this local temporal context continues across forecast calls.

The selective parameters are generated from u_t by learned linear projections:

$$[r_t, B_t, C_t] = W_x u_t, \quad r_t \in \mathbb{R}^{d_t}, \quad B_t, C_t \in \mathbb{R}^{d_{\text{state}}}, \quad (5)$$

$$\Delta_t = \text{softplus}(W_{\Delta} r_t + b_{\Delta}) \in \mathbb{R}^{d_{\text{inner}}}. \quad (6)$$

Here $d_t = 32$ and $d_{\text{state}} = 16$. The projection matrices, convolution kernels, and biases are trainable parameters optimized using the forecasting loss. In contrast, B_t , C_t , and Δ_t are input-dependent values generated anew at each time step from u_t . The softplus function ensures $\Delta_t > 0$.

The continuous-time state matrix is diagonal and parameterized as

$$A_{d,n} = -\exp(A_{\log,d,n}), \quad d = 1, \dots, d_{\text{inner}}, \quad n = 1, \dots, d_{\text{state}}. \quad (7)$$

Because every element of A is negative, the corresponding factor $\exp(\Delta_{t,d} A_{d,n})$ lies between zero and one, giving stable, decaying memory. The implementation uses the following selective recurrence:

$$h_{t,d,n} = \exp(\Delta_{t,d} A_{d,n}) h_{t-1,d,n} + \Delta_{t,d} B_{t,n} u_{t,d}. \quad (8)$$

The first term retains a scaled portion of the previous state, while the second writes the current input into memory. A small $\Delta_{t,d}$ gives slow decay and long memory; a large $\Delta_{t,d}$ gives faster decay and a stronger update from the current input.

The state is read out using the selective vector C_t and a learned skip parameter D :

$$s_{t,d} = \sum_{n=1}^{16} C_{t,n} h_{t,d,n} + D_d u_{t,d}. \quad (9)$$

Finally, the gate branch z_t modulates this readout, and the result is projected back to the W_{GC} -dimensional GraphCast latent space:

$$o_t = W_{\text{out}} [s_t \odot \text{SiLU}(z_t)], \quad x_t^{\text{out}} = x_t^{\text{in}} + o_t. \quad (10)$$

The principal experiments use $d_{\text{inner}} = 16$, an SSM state dimension of 16 per channel, a causal convolution width of 4, and a rank-32 projection for Δ_t ; we also examine $d_{\text{inner}} = 32$ and 64. The SSM state h_t and the three-token convolution cache persist separately for each batch element, mesh node, and Mamba layer across autoregressive forecast steps. The equations above describe one shared group of input-dependent B_t, C_t coefficients. The grouped variant partitions the inner channels into groups with separate B_t, C_t coefficients; the memory ablation compares one and four such groups.

An additive correction with internal temporal memory. Let X_t denote the two-frame meteorological input and the associated forcings. The forecast combines the frozen baseline G_θ with a trainable correction R_ϕ :

$$(c_t, h_t) = R_\phi(X_t, h_{t-1}), \quad \hat{Y}_t = G_\theta(X_t) + c_t, \quad \theta \text{ fixed}. \quad (11)$$

The correction branch has its own grid-to-mesh encoder, mesh processor, and mesh-to-grid decoder. It receives the same meteorological inputs as the baseline, rather than the baseline’s latent features. Its mesh processor uses two message-passing steps, each followed by a two-layer Mamba module. Schematically,

$$L_t^{(0)} = \text{Encode}_\phi(X_t), \quad (12)$$

$$\tilde{L}_t^{(k)} = \text{GNN}_\phi^{(k)}(L_t^{(k-1)}), \quad (13)$$

$$(L_t^{(k)}, h_t^{(k)}) = \text{TemporalModule}_\phi^{(k)}(\tilde{L}_t^{(k)}, h_{t-1}^{(k)}), \quad k = 1, 2, \quad (14)$$

$$c_t = \text{Decode}_\phi(L_t^{(2)}). \quad (15)$$

Each temporal module contains the residual Mamba updates described above; the state notation includes both SSM states and convolution caches. The output correction is initialized to zero. This formulation is additive at the forecast output and uses temporal processing within the residual GNN branch. During autoregressive forecasting, the combined prediction is fed into the next input, and the temporal states carry information from the preceding trajectory.

2 Numerical results

We focus on 1° GraphCast-small models and ask two questions: does a learned correction improve long-range forecast skill, and does persistent temporal memory contribute beyond the additional trainable capacity? We first report forecast skill, then isolate the effect of state carry, and finally examine sensitivity to training choices.

2.1 Evaluation protocol and forecast loss

The matched-start evaluations use 32 forecast initializations from 2022 and 40 six-hour forecast steps, corresponding to ten days. Forecasts start with zero temporal state and use the full combined prediction as feedback, without a history warm start. The pretrained GraphCast baseline remains frozen during correction training. The principal correction branch has spatial width 512, two message-passing steps, and Mamba inner dimension 16.

Our primary metric is the original GraphCast normalized weighted mean-squared error, including its difference-standard-deviation normalization, latitude weights, pressure-level weights, and variable weights. If $\ell_{i,k}^{\text{model}}$ and $\ell_{i,k}^{\text{GC}}$ are the losses for start i and lead k , the rollout loss reduction is

$$\mathcal{I}_{1:40} = 100 \left(1 - \frac{\sum_{i=1}^{32} \sum_{k=1}^{40} \ell_{i,k}^{\text{model}}}{\sum_{i=1}^{32} \sum_{k=1}^{40} \ell_{i,k}^{\text{GC}}} \right). \quad (16)$$

Day-ten reduction uses only $k = 40$. We aggregate losses before taking the ratio; these percentages are neither RMSE reductions nor arithmetic averages of per-lead percentages. Each evaluated model uses its saved matched baseline. The older reset study reconstructs an approximate loss from channel RMSEs and is explicitly identified as such below.

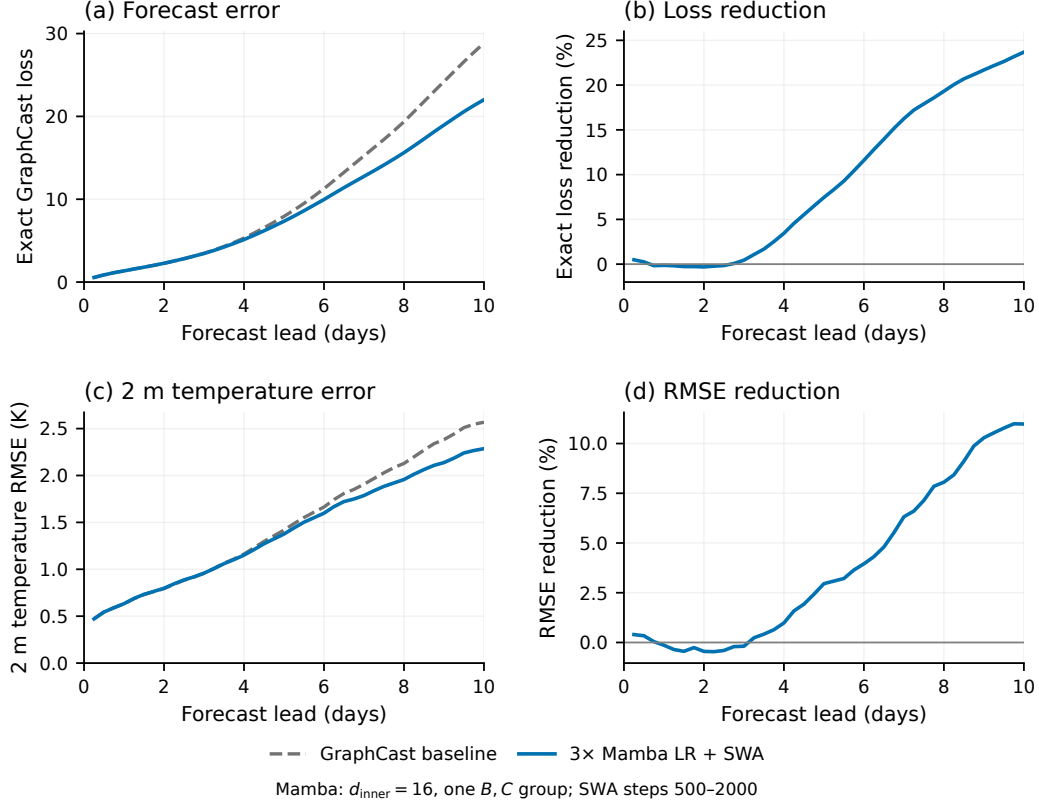


Figure 2: Forecast skill of the $d_{\text{inner}} = 16$ correction trained with a threefold Mamba learning rate and stochastic weight averaging (SWA) at 1° on 32 matched starts. Top: exact GraphCast loss and per-lead loss reduction. Bottom: cosine-latitude-weighted 2 m temperature RMSE (K) and RMSE reduction on the same starts. Reductions are relative to its paired GraphCast baseline. Training and model-selection details are given in Appendix B.

2.2 Improved skill at longer forecast leads

Figure 2 shows the $d_{\text{inner}} = 16$ correction trained with a $3\times$ Mamba learning rate and SWA. It reduces exact aggregate ten-day loss by 16.34% and day-ten loss by 23.68%. The per-lead reduction grows from 3.45% at day four to 16.25% at day seven and 23.68% at day ten. Day-one loss reduction is -0.13% , indicating a small degradation; short-lead skill is largely preserved rather than uniformly improved. At day ten, 2 m temperature RMSE is 2.29 K, a 10.98% reduction relative to the paired baseline.

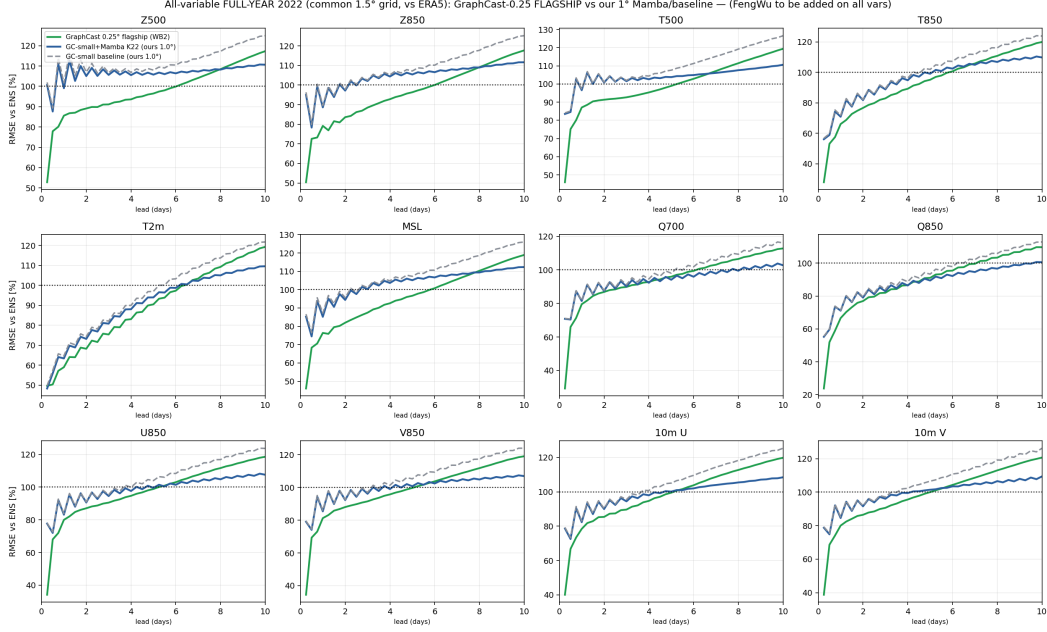


Figure 3: Complete common-grid comparison. The quantities span geopotential, temperature, sea-level pressure, humidity, and wind. Blue is the archived K22 correction, dashed gray its GraphCast-small baseline, and green the 0.25° reference. Forecasts are verified on a common 1.5° grid, with RMSE normalized to the displayed ENS reference (100% denotes equal RMSE). This historical comparison is separate from the matched-start evaluations in Figure 2.

We further assess forecast skill for the meteorological quantities in Figure 3, considered in the original GraphCast study Lam et al. [2023]. These panels compare forecasts on a common 1.5° verification grid and again show more favorable error growth with forecast lead time for our corrected 1° model than for its uncorrected baseline. Note that the model output resolution and verification resolution are different quantities. Remarkably, in this comparison, the corrected 1° model achieves lower errors at longer leads even than the 0.25° GraphCast-large reference, despite its coarser spatial resolution and the substantial computational investment in training the reference model: approximately four weeks on 32 TPU v4 devices Lam et al. [2023].

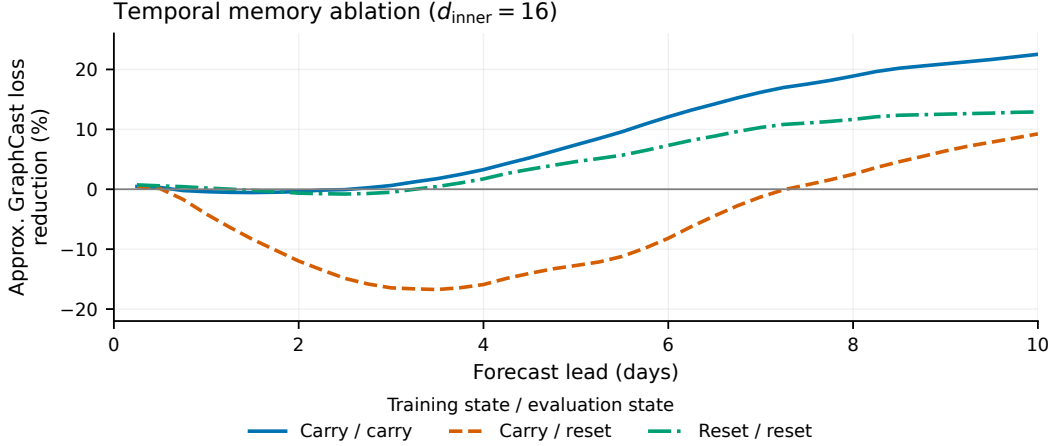


Figure 4: Persistent memory improves long-lead forecasts. All three policies use the same architecture with $d_{\text{inner}} = 16$, one B, C group, and SWA over updates 2k–8k. Carry/carry retains state during training and forecasting; carry/reset clears state during forecasting; reset/reset clears it during both. Carry/carry achieves the largest day-ten loss reduction. Results use 32 matched cold starts and 40 six-hour leads; positive values indicate lower loss than the paired GraphCast baseline. Loss is reconstructed approximately from saved channel RMSEs.

2.3 Evidence for useful temporal memory

A correction branch can improve forecasts through additional spatial capacity as well as temporal memory. To distinguish these contributions, we compare three policies: state carry during both training and evaluation; state carry during training followed by resetting at every evaluation step; and a separately trained model with resetting at every training and evaluation step. Resetting zeros both the SSM state and the convolution cache. The carry-trained and reset-trained runs have the same architecture, data settings, optimizer, and training seed; their temporal-state policy differs.

Figure 4 isolates the temporal-state policy for a fixed $d_{\text{inner}} = 16$ model. Carry/carry gives the strongest long-lead improvement: day-ten approximate loss reduction is 22.52%, compared with 9.24% for carry/reset and 12.91% for reset/reset. Retaining memory therefore adds 13.28 percentage points relative to resetting the trained state at inference and 9.62 percentage points relative to training and evaluating without persistent state. Additional group settings and averaging windows are reported in Appendix A.

The inference-time reset demonstrates dependence on the learned state, but also introduces a train–evaluation mismatch. The separately trained reset control is therefore essential: it retains the trainable correction architecture while removing persistent temporal context. Its smaller long-lead benefit supports a contribution from memory beyond additional correction capacity. This is evidence for useful history dependence relative to the baseline’s two-frame input representation. The experiment does not identify a unique physical memory kernel or isolate the SSM contribution from that of the convolution cache.

2.4 Sensitivity to inner dimension and temporal learning rate

Table 1 compares inner dimensions 16 and 32 under two training methods: joint learning rates, which use equal learning rates for the spatial and Mamba parameters of the correction branch, and a $3\times$ Mamba learning rate with the spatial learning rate unchanged. All four configurations use the same training schedule and SWA window. Under joint learning rates, the dimension-32 correction achieves larger aggregate and day-ten loss reductions than the dimension-16 correction. With a $3\times$ Mamba learning rate, the ordering reverses: the dimension-16 correction shown in Figure 2 achieves the largest reductions in the table. The benefit of increasing inner dimension therefore depends on the temporal learning rate in these runs. Each run starts from its own trained warm-up parameters, so these are comparisons of complete training runs. Appendix B gives the training details and the additional $0.3\times$ Mamba learning-rate results.

Table 1: Inner dimension and Mamba learning-rate comparison at 1° on the matched 32-start development evaluation. Entries are exact GraphCast loss reductions (%); bold marks the largest value in each metric column. All runs use spatial width 512, one B, C group, batch size four, the same training schedule, and SWA over branch updates 500, 1,000, 1,500, and 2,000. The spatial learning rate is unchanged; $1\times$ denotes joint learning rates.

d_{inner}	Mamba LR multiplier	Ten-day aggregate	Day ten
16	$1\times$ (joint)	15.67	22.44
32	$1\times$ (joint)	16.29	23.32
16	$3\times$	16.34	23.68
32	$3\times$	15.73	22.75

Scope of the evidence. The exact-loss studies use one training seed and a 32-start development set used in model selection. We therefore report descriptive improvements without significance claims. The memory controls support the contribution of persistent state in the tested configuration; they do not directly establish the same attribution for every model. Evaluation on forecast dates reserved from selection, with matched uncertainty estimates and exact-loss reset controls, would strengthen the quantitative conclusions.

References

- Xander M De Wit, Alessandro Gabbana, Michael Woodward, Yen Ting Lin, Federico Toschi, and Daniel Livescu. Data-driven mori–zwanzig modeling of lagrangian particle dynamics in turbulent flows. *Proceedings of the National Academy of Sciences*, 123(13):e2525390123, 2026.
- D. A. Donzis, P. K. Yeung, and K. R. Sreenivasan. Dissipation and enstrophy in isotropic turbulence: Resolution effects and scaling in direct numerical simulations. *Physics of Fluids*, 20(4):045108, 04 2008. ISSN 1070-6631. doi: 10.1063/1.2907227. URL <https://doi.org/10.1063/1.2907227>.
- Albert Gu and Tri Dao. Mamba: Linear-time sequence modeling with selective state spaces. *arXiv preprint arXiv:2312.00752*, 2023.
- Pavel Izmailov, Dmitrii Podoprikin, Timur Garipov, Dmitry Vetrov, and Andrew Gordon Wilson. Averaging weights leads to wider optima and better generalization, 2019. URL <https://arxiv.org/abs/1803.05407>.
- Remi Lam, Alvaro Sanchez-Gonzalez, Matthew Willson, Peter Wirnsberger, Meire Fortunato, Ferran Alet, Suman Ravuri, Timo Ewalds, Zach Eaton-Rosen, Weihua Hu, Alexander Merose, Stephan Hoyer, George Holland, Oriol Vinyals, Jacklynn Stott, Alexander Pritzel, Shakir Mohamed, and Peter Battaglia. Learning skillful medium-range global weather forecasting. *Science*, 382(6677):1416–1421, 2023. doi: 10.1126/science.adi2336. URL <https://doi.org/10.1126/science.adi2336>.
- Yen Ting Lin, Yifeng Tian, Daniel Livescu, and Marian Anghel. Data-driven learning for the mori–zwanzig formalism: A generalization of the koopman learning framework. *SIAM Journal*

on Applied Dynamical Systems, 20(4):2558–2601, 2021. doi: 10.1137/21M1401759. URL <https://doi.org/10.1137/21M1401759>.

John D. Wilson and Brian L. Sawford. Review of lagrangian stochastic models for trajectories in the turbulent atmosphere. *Boundary-Layer Meteorology*, 78(1):191–210, 1996. ISSN 1573-1472. doi: 10.1007/BF00122492. URL <https://doi.org/10.1007/BF00122492>.

Xiang IA Yang and Kevin P Griffin. Grid-point and time-step requirements for direct numerical simulation and large-eddy simulation. *Physics of Fluids*, 33(1), 2021.

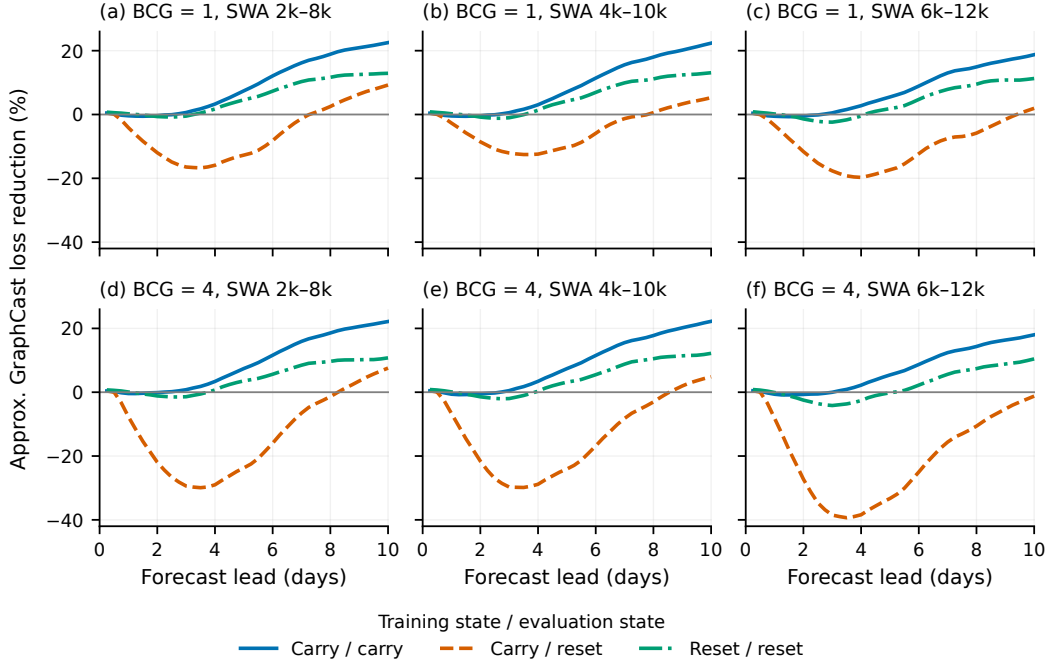


Figure 5: Complete temporal-state comparison. Column titles specify the SWA windows, and rows specify one or four B, C groups. All panels use approximate GraphCast loss reconstructed from saved channel RMSEs.

A Memory ablation across averaging windows

The full reset study fixes $d_{\text{inner}} = 16$ and varies the number of B, C groups (one or four) and the SWA window (2k–8k, 4k–10k, or 6k–12k updates). Carry/carry achieves the largest day-ten reduction in each panel of Figure 5. The SWA windows overlap and are derived from the same training trajectories; they are sensitivity checks, not independent seed repetitions. The approximate metric and the distinct train/evaluation reset policies are the same as in Section 2.3. Each curve uses its saved baseline (maximum baseline channel-RMSE difference across the full study: 0.56%); exact pole weighting cannot be recovered from these channel summaries.

B Training choices and checkpoint averaging

Training methods and model selection. Table 1 reports four SWA configurations from the temporal learning-rate study: joint learning rates and a $3\times$ Mamba learning rate, each with inner dimensions 16 and 32. Joint learning rates apply the same rate to the spatial and Mamba parameters of the correction branch; the $3\times$ configuration increases only the Mamba rate. The pretrained GraphCast parameters remain frozen in all cases. Figure 2 uses the dimension-16, $3\times$ Mamba-LR SWA configuration, which has the largest aggregate and day-ten exact-loss reductions among the completed SWA configurations in the reported optimizer and learning-rate studies. This selection uses the matched 32-start development evaluation and does not establish optimality on an independent test set.

Optimizer study. The optimizer study varies peak learning rate (10^{-4} or 5×10^{-5}), Adam moments, temporal initialization scheme (legacy or Mamba1), and the paired inner-dimension/group settings (16/1 or 64/4). The latter changes two factors simultaneously and does not isolate inner dimension. Runs use 10,000 updates, seed 22, a 200-update warm-up, cosine decay to one tenth of the peak learning rate, and weight decay 10^{-4} . The correction is trained with closed-loop feedback, 24-step backpropagation windows, a 20-step autoregressive tail, and loss across all 24 steps. The correction contribution to the feedback is stop-gradient, while temporal-state gradients propagate within the backpropagation window.

Each run screens periodic checkpoints and three SWA windows on eight forecast starts. The selected checkpoint and SWA are then evaluated on the matched 32 starts. Selection uses these development evaluations, so the study does not represent an untouched test-set comparison.

Day-ten reductions for the 22 completed selected SWA configurations range from 13.47% to 23.21%. The configuration at the upper end uses Mamba1 initialization, inner dimension 16, one B, C group, learning rate 10^{-4} , and AdamW moments (0.9, 0.98); its aggregate loss reduction is 16.28%.

Temporal learning rate and inner dimension. The temporal learning-rate study keeps spatial width 512 and one B, C group while varying inner dimension and the Mamba learning-rate multiplier. Each arm uses its own 500-update warm-up followed by 2,000 branch updates at batch size four. The base peak learning rate is 10^{-4} with AdamW moments (0.9, 0.98). Table 2 extends the four configurations in Table 1 with the $0.3\times$ Mamba learning rate at each inner dimension. All six use SWA over branch updates 500, 1,000, 1,500, and 2,000. The warm-up parameter values differ across arms despite the common training seed, so the comparisons describe complete training runs rather than isolated learning-rate interventions.

Under joint learning rates, the dimension-32 SWA correction exceeds the dimension-16 correction’s aggregate loss reduction by 0.62 percentage points. Under the $3\times$ Mamba learning rate, the dimension-32 correction instead has a smaller aggregate reduction by 0.61 percentage points. The $0.3\times$ Mamba-LR SWA results also favor dimension 32, while each dimension attains its largest reductions at a different multiplier: $3\times$ for dimension 16 and $1\times$ for dimension 32. These observations do not establish a monotonic benefit from increasing inner dimension or temporal learning rate.

Table 2: Exact GraphCast loss reduction (%) for the learning-rate study’s SWA models. Spatial learning rate is unchanged across these six arms; the multiplier applies to Mamba parameters. All evaluations contain the same 32 starts.

d_{inner}	Mamba LR multiplier	Ten-day aggregate	Day ten
16	$0.3\times$	14.70	21.98
16	$1\times$	15.67	22.44
16	$3\times$	16.34	23.68
32	$0.3\times$	15.82	22.96
32	$1\times$	16.29	23.32
32	$3\times$	15.73	22.75

Stochastic weight averaging. SWA averages model parameter values from saved checkpoints to reduce dependence on a single training iterate Izmailov et al. [2019]. Every configuration in Tables 1 and 2 averages four checkpoints at branch updates 500, 1,000, 1,500, and 2,000. The dimension-16 Mamba1-initialized optimizer-study configuration described above averages seven checkpoints, saved every 1k updates from 2k through 8k. These are parameter averages, not ensembles of independent forecast predictions. Evaluation starts temporal state from zero, including for SWA models.